ADVANCED WEB DEVELOPMENT FRAMEWORKS (ITUE301)

Practical 9: In-Memory Caching and Query Optimization

CO/PO	CO2, CO4 / PO3, PO5
Objective	To implement server-side caching and measure its impact on API response time.
Prerequisites	• Practicals 4–7 completed working Node/Express/MongoDB backend with authentication • Basic understanding of why repeated database reads are expensive at scale

Theory Concepts (Self-Study Reference)

Coursera Pointer	See Week 9 IBM: Node.js & MongoDB Developing Back-end Database Applications, Module 5 (optimization techniques, back-end performance, scaling approaches, load balancing overview).

Architecture / Diagram

Diagram

```
GET /tasks request
        |
        v
  Cache check (node-cache)
    ├── HIT  → return cached data immediately
    └── MISS → query MongoDB → store in cache → return data

POST/PUT/DELETE /tasks
        |
        v
  Write to MongoDB → invalidate cache key
```

Problem Definition

Problem Statement	• Add in-memory caching to the Task Management backend using node-cache. • Cache the response of GET /tasks (all tasks) with a reasonable TTL (e.g., 60 seconds). • Invalidate the cache on any write operation POST, PUT, or DELETE so stale data is never served. • Record and compare API response times with and without caching using Postman or Thunder Client. • Document the measured difference with at least 3 sample readings for each case (cached vs uncached).
Key Questions / Analysis	• Why must the cache be invalidated on every write operation, and what would happen to data correctness if it were not? • What is a reasonable TTL (time-to-live) for cached data in a task management context, and what trade-off does TTL length represent? • Why is in-memory caching (node-cache) not suitable for a multi-server/multi-instance deployment, even though it works fine in this lab?
Supplementary Problems	• Cache the GET /tasks/:id single-task endpoint separately from the all-tasks endpoint. • Add a cache-hit/cache-miss counter and expose it via a debug endpoint. • Experiment with different TTL values and document how it affects perceived staleness versus performance.

Key Skills Addressed	• Implementing in-memory caching using node-cache • Designing correct cache invalidation logic for write operations • Measuring and comparing API response times empirically • Reasoning about the trade-offs of caching strategies
Learning Outcome	Students will be able to implement in-memory caching for a REST API, ensure cache correctness through invalidation, and measure the performance impact using real response time data.
Lab Session Step-by-Step	

Steps	1. Install node-cache:
	`npm install node-cache`
	2. Initialize a cache instance in a shared module:
	`const NodeCache = require('node-cache');`
	`const cache = new NodeCache({ stdTTL: 60 });`
	`module.exports = cache;`
	3. Update the GET /tasks route to check the cache first:
	`const cached = cache.get('all_tasks');`
	`if (cached) return res.json(cached);`
	`const tasks = await Task.find();`
	`cache.set('all_tasks', tasks);`
	`res.json(tasks);`
	4. Invalidate the cache inside POST, PUT, and DELETE handlers after a successful write:
	`cache.del('all_tasks');`
	5. In Postman, send GET /tasks 3 times and record response time for each (look at the "Time" field).
	6. Temporarily disable caching (comment out the cache check) and repeat the same 3 requests for comparison.
	7. Record both sets of readings in a simple table for the lab journal.

Tools / Technology / Resources Required	• Node.js (v18+), npm • node-cache • Express.js, Mongoose • Postman or Thunder Client (for response time measurement)
Total Hours	• Lab Session: 2 hours • Implementation: 1.5 hours • Testing / Modification: 0.5 hours • Total Engagement: 2 hours
Faculty Teaching Guide	
Guidance Notes	• Tell students explicitly to record uncached response times BEFORE adding the cache check, or the comparison cannot be done fairly afterward. • Clarify that node-cache is process-local memory explain briefly why this would not work correctly if the app ran across multiple server instances (sets up the later Unit 4 scaling discussion). • Walk through a live example of forgetting cache invalidation, showing students a stale response after an update, to make the risk concrete. • Encourage at least 3 repeated readings per condition rather than a single measurement, since response time can vary.
Common Mistakes	• Forgetting to invalidate the cache after a write, leading to stale data being served indefinitely within the TTL window. • Caching per-request without a stable cache key, causing every request to be treated as a cache miss. • Only taking a single response time reading instead of multiple samples, making the comparison unreliable. • Setting a TTL so short that the cache effectively never gets hit during testing.
Troubleshooting Guide	

Symptom	Likely Cause	Fix
Updated task not reflected in GET response	Cache not invalidated after PUT/DELETE	Add cache.del('all_tasks') inside every write route handler
No measurable difference between cached and uncached	TTL too short or cache key changing on every request	Use a fixed, consistent cache key string and a TTL of at least 60 seconds for testing
Cache works but server restarts wipe it	node-cache is in-memory and resets on restart	This is expected behavior for in-memory caching note it in the lab journal observation as a known limitation
Response time barely changes	MongoDB query was already fast (small dataset)	Add more sample task documents to the database to make the cache benefit more visible

Student Assignment / Post Laboratory Work

Tasks	• Review the Theory Concepts section and the assigned Coursera module before next week. • Cache the single-task GET /tasks/:id endpoint separately from the all-tasks endpoint. • Add a simple cache-hit/cache-miss counter exposed via a debug endpoint.

GitHub Deliverables

Expected Repository Contents	• Same repository as Practical 4/5, updated with new commits • node-cache implementation with correct invalidation on all write operations • Response time comparison data (cached vs uncached) included in README or docs folder • At least one commit specifically for caching implementation

Rubrics (Practical Evaluation / Viva)

Criteria (Marks)	Description
Conceptual understanding (2 Marks)	Explains what and why, not just what was done
Implementation (4 Marks)	Code written, runs, and is original
Correctness (2 Marks)	Output matches expected result
Reflection (1 marks)	Errors documented and explained
Lab file evidence (1 marks)	Completeness and presentation
Total: 10 marks	**Passing Threshold: 5 / 10 (50%)**